

Digstore ### The Content-Addressable WASM Store Format

Michael Taylor

Version 2.1 · June 2026

About this revision (v2.0). This edition reconciles the specification with the reference implementation after a security audit and hardening pass. Where v1.0 described a design that the implementation has since corrected, this edition describes the corrected design and says so. Notably: the chunk cipher is **AES-256-GCM-SIV** (not plain GCM with a fixed nonce); clone/pull now verify a **publisher signature over the served root** ("authenticated head"); the merkle leaf is a **per-resource ciphertext digest** (not a per-chunk hash); the data-section codec is **big-endian**; and several guarantees that were overstated in v1.0 (execution proofs, JWT authorization) are now stated with their true, current scope. See §24, Changes from v1.0 and §22, Security Considerations and Residual Risks.

What's new in v2.1. The `.dig` directory is now a multi-store **workspace** (named stores under `stores/<name>/`, §4.4); each store carries a per-store **128 MB content cap** and an explicit **content root** for build output; size-obfuscation filler is now **uniform** so every module compiles to one fixed size regardless of content (§8.3); and the module memory ceiling is raised to a configurable **384 MiB** with the guest heap placed dynamically above the data section (§5.1, §18.2).

Abstract

Digstore is a self-contained, content-addressable, encrypted-at-rest store format. A Digstore store **is** a WebAssembly module. Store content compiles into a single module file — a WebAssembly binary distributed with the `.dig` extension — whose data section embeds the chunked, encrypted content, the merkle commitments, the root history, the store's public key, and a set of trusted-host keys. The module exposes a fixed export ABI. A host runtime instantiates it in a sandbox and retrieves content by executing it. The artifact is executable, not passive: the content and the logic that serves it ship as one binary.

A store module gates its own access. Before releasing content it runs an attestation handshake against the host, establishes a session, and (where configured) applies a JWT authorization gate. Each response is bound to the store's trusted root by a merkle inclusion proof and to the URN by authenticated encryption; a serving node can additionally attach a zero-knowledge proof that the response is the correct result of the deterministic serving computation, verified against the module's hash. The module embeds no secret of any kind.

Files are addressed by URNs of shape `urn:dig:<chain>:<storeID>[:<rootHash>][/<resourceKey>]` . Content is chunked with content-defined chunking, hashed with SHA-256, committed in a per-generation merkle tree, and encrypted at rest with a key derived from the URN itself. Invalid URNs do not error: the module returns deterministic decoy bytes drawn from a logarithmic size distribution, so a host cannot trivially distinguish a real lookup from a miss.

A store is identified by a 32-byte store ID equal to SHA-256 of the store's BLS public key, so the identifier is self-certifying. Every commit produces a new module file named `{storeID}-{rootHash}.dig` , computes a new merkle root, and appends it to the root history. The module is the single distribution unit: one portable executable that is both the data and the server.

The developer-facing workflow is Git: `init` , `add` , `commit` , `log` , `diff` , `checkout` , and `clone` behave as a developer expects. A commit is a generation and its root hash is the commit identifier. The chunking, the URN-derived encryption, the merkle commitments, and the compile-to-WebAssembly step all run beneath that familiar surface, so the cryptography is a property of the format rather than a task in the workflow.

The privacy consequence is the heart of the format. The two values that turn a URN into content are both derived from the URN and nothing else: the retrieval key that locates a resource is a hash of the URN, and the decryption key is derived from the URN by HKDF-SHA256. A provider serving a Digstore module (a DIG Node acting as a neutral pipe) holds an opaque executable and ciphertext. It never sees the URN. Without the URN it cannot compute the retrieval key for any resource it does not already know, cannot derive a decryption key, and cannot read what it serves. Decryption runs on the client after it receives the encrypted chunk, never on the provider.

This paper covers the store entity, the WASM module format and its compilation pipeline, the host/module ABI, the URN system, the content model, the encryption and zero-knowledge schemes, the attestation and execution-proof systems, the host runtime, and the command-line interface. It documents the local store format and its HTTPS remote protocol. Network distribution across many providers, external identity anchoring, and payment settlement are out of scope.

Table of Contents

Part 1: Foundation — 1. The Big Ideas · 2. Positioning · 3. Design Heritage

Part 2: The WASM Store Format — 4. Store Structure · 5. The Store Module · 6. The Host/Module ABI · 7. URN System · 8. Content Model · 9. Merkle Proofs

Part 3: Security and Zero-Knowledge — 10. Threat Model · 11. URN-Based Encryption · 12. Host Attestation and Sessions · 13. Execution Proofs · 14. Oblivious Retrieval · 15. Provider Blindness and the Neutral Pipe · 16. Temporal Keys · 17. The Secretless Module

Part 4: Runtime and Tooling — 18. The Host Runtime · 19. Compilation · 20. Developer Experience · 21. Remotes: Push and Pull over HTTPS

Part 1: Foundation

1. The Big Ideas

1. Every URN is a key. Retrieval key, encryption key, access token: all derived from the URN and nothing else. The retrieval key that locates a resource is `SHA-256(canonical_urn)`. The decryption key is `HKDF-SHA256` of the canonical URN. Anyone holding the URN can locate and decrypt; anyone lacking it can do neither. There is no separate access-control list — the URN is the credential.

2. The store is an executable. A Digstore store compiles to a WebAssembly module. The content lives in the module's data section; the logic to authenticate a host, gate access, and serve content lives in the code section. The host does not parse the module — it runs it inside a sandbox through a fixed export ABI. A store that wants to enforce authorization carries that enforcement inside itself.

3. Content is provably genuine; execution is provably correct (within a stated scope). A response is bound to the store's trusted root by a merkle inclusion proof and to the URN by authenticated encryption, so no host can make a client accept fabricated, substituted, or truncated content (§9.4, §11). When fetched from a remote, the served root is additionally verified to carry the publisher's BLS signature, so a relay cannot present content the publisher never authorized (§21.6, "authenticated head"). On top of that, a serving node can attach a zero-knowledge proof that the response is the correct output of the deterministic serving computation for the genuine module, verified against the module's hash (§13). The proof carries no secret. *The current proof circuit re-executes the serving computation (key lookup, ciphertext gather, output commitment); it binds the program hash as an identifier rather than constraining WASM-opcode execution — see §13.1.*

4. Invalid URNs are indistinguishable from valid URNs. A host serving requests cannot trivially tell which URNs map to real content. Invalid URNs return deterministic decoy bytes whose size is drawn from a logarithmic distribution seeded by the URN. Valid URNs return ciphertext plus a proof. Both look alike on the wire; enumeration degrades to guessing.

5. The provider is blind. Retrieval and decryption keys are both functions of the URN, so a provider that serves a module learns nothing from doing so. It receives a retrieval key (a hash) and returns ciphertext. It never holds the URN, so it cannot reverse the hash, cannot derive the decryption key, and cannot inspect the content it relays. A DIG Node is a neutral pipe by construction, not by policy.

The module is the source of truth for content. A 32-byte store ID names the store; the current root hash names its state.

2. Positioning

A conventional content store is a passive container: a host parses it, serves raw bytes, and all access logic lives in the client. Digstore puts that logic into the artifact. The store is an executable that defends itself — it decides whom to serve, decrypts only what a valid request names, and binds what it returns to a signed root.

Capability	Passive content store	Digstore WASM module
Artifact	File parsed by the host	Executable module run by the host
Served by host as	Raw bytes	Result of a sandboxed execution
Access control	Client-side, by possession	In-module: attestation + session + optional JWT gate
Response integrity	Merkle proof against root	Merkle proof + authenticated-head signature + optional ZK execution proof
Content addressing	Varies	SHA-256 throughout; URN-addressed
Encryption at rest	Out-of-band or none	URN-derived AES-256-GCM-SIV, embedded in module
Invalid lookups	Error / 404	Deterministic decoy, logarithmic size distribution
Byte-range retrieval	Application-specific	Resource key + range, served by module
Distribution unit	One container file	One <code>{storeId}-{rootHash}.dig</code> file

What the Digstore module delivers:

- **Self-enforcing access.** Authorization checks run inside the module. A host cannot serve content the module would have refused, because the host does not hold the decryption logic — it holds an opaque executable.
- **Provenance per response.** A merkle proof binds the response to the trusted root; on a remote, the served root is verified against the publisher's signature; and an optional ZK proof attests the serving computation. A relay cannot substitute, truncate, or fabricate content without detection, and there is no secret it could extract to forge one.
- **Host attestation.** A module refuses to operate until the host proves its identity against trusted keys baked in at compile time.
- **Provider blindness.** Retrieval and decryption keys derive from the URN. A provider relays ciphertext addressed by a hash and never sees the URN.
- **A Git-shaped interface.** `init`, `add`, `commit`, `log`, `diff`, `checkout`, `clone` — the verbs and mental model Git already taught.
- **One portable executable.** The store is a single `.dig` file (a WebAssembly module). Copying it is a backup; running it is a server.

What the WASM store deliberately does not do: no working tree (content is read by URN through execution); no branching (linear root history, one module + one root per commit); no partial module (the whole store is one module); no host-side decryption (the host runs the module, it does not parse content out of it).

3. Design Heritage

Digstore borrows, with attribution.

Component	Heritage	Digstore adaptation
Content addressing	Git, IPFS	SHA-256 throughout; URN scheme distinct from CID
Executable data store	WebAssembly, eBPF, capability machines	Content + access logic compiled to a sandboxed WASM module
Sandboxed execution	wasmtime, wasmer	Host runtime with timeout, fuel, and memory/table bounds; fixed import/export ABI
Content-defined chunking	rsync, restic, borg, FastCDC	Gear-based rolling hash; target 64 KiB, min 16 KiB, max 256 KiB
Merkle commitment	Git, Ethereum, Plasma	Per-generation tree over per-resource ciphertext leaves; root is the generation root hash
Stream-cipher-from-hash / KDF	NaCl, age, libsodium	HKDF-SHA256 from URN → AES-256-GCM-SIV key
URN format	RFC 8141	<code>urn:dig:<chain>:<storeID>[:<rootHash>] [/<resourceKey>]</code>
Code obfuscation	Software-protection literature	Deterministic instruction substitution, opaque predicates, bogus code, control-flow nops (security never rests on it)
Host attestation	TEE remote attestation	BLS attestation against compile-time trusted keys
Execution proofs	zkVM / proof-carrying code (zk-STARK, SNARK)	Zero-knowledge proof of the deterministic serving computation; no embedded secret

What Digstore does **not** inherit: Git's object graph (no trees/blobs/commits as objects — a generation is a flat chunk set with a merkle tree) and IPFS's DHT (location is determined out-of-band, not by a content router).

Part 2: The WASM Store Format

4. Store Structure

A store is a unified entity tying together a 32-byte store ID, a WASM module, and local configuration. In code this is the `Store` entity (`digstore-store`), wrapping the compiled module and a `StoreConfig` .

4.1 Store Configuration

```
pub struct StoreConfig {
    pub store_id: Bytes32,           // 32-byte store identifier
    pub data_dir: String,           // local working directory
    pub max_size: u64,              // maximum store size in bytes
    pub visibility: Visibility,      // Public (URN decrypts) | Private (URN + salt)
}

pub enum Visibility {
    Public,                        // default: anyone holding the URN can decrypt
    Private(SecretSalt),          // URN alone is not enough; salt held by publisher
}
```

No absolute paths leak into addressing. The `visibility` field is the publisher's access-model choice, made once at `init` and fixed for the store's life. A public store is readable by anyone holding the URN; a private store stays sealed even to a URN-holder without the salt (§11.4). Per-store policy (such as JWT authentication) is likewise carried as configuration and compiled into the module, not enforced by the host.

On-disk protection. When `visibility = Private` , `config.toml` embeds the `SecretSalt` — master key material. The CLI writes `config.toml` , the BLS signing-key seed (`signing_key.bin`), and the salt file with owner-only permissions (`0600` on Unix; the user-profile ACL on Windows).

4.2 Store ID

32 bytes, hex-encoded (64 characters). The store ID is **SHA-256 of the store's BLS public key** (§20.1), which makes it self-certifying: any party that learns a store ID can later check that a module claiming that ID embeds a public key whose hash equals it. It is carried into every URN that references the store and is the only mandatory addressing component. Anchoring the store ID to an external identity is out of scope for this document.

4.3 Generations and Root Hash

Term	Meaning
Generation	A store state identified by a root hash, with an ordinal <code>GenerationId</code> (u64)
Root hash	Merkle root over the generation's per-resource leaves (<code>GenerationState.root</code>)
Root history	Append-only list of every root hash the store has produced

```

pub struct GenerationState {
    pub id: GenerationId, // monotonic u64
    pub root: Bytes32,    // merkle root of this generation
    pub timestamp: u64,    // unix seconds
}

pub struct Generation {
    pub state: GenerationState,
    pub tree: MerkleTree, // tree over the generation's per-resource leaves
}

```

Each commit produces exactly one new generation, one new module file, and one new root hash. The root history grows monotonically and is exported by the module via `get_roothash_history`. Any past root hash can be quoted in a URN to address the store at that generation.

4.4 On-Disk Layout

A project lives in a `.dig` directory, the way a Git repository lives in `.git`. `.dig` is a **workspace** that holds one or more named stores under `stores/<name>/`. `digstore init [name]` creates `.dig` in the current working directory (defaulting the store name to `default`); every other command discovers the workspace by walking up from the current directory to the nearest `.dig` (an explicit `--dig-dir` overrides this). So the CLI operates on the workspace that contains the directory you run it from.

```

<project>/.dig/
  workspace.toml           # active store + name → store_id registry + per-store content
  stores/
    <name>/
      {store_id}.staging.bin # one full store per name
      generations/
        {roothash_hex}/
          manifest.json      # generation metadata
          chunks/{chunk_hash_hex} # one file per unique chunk
      modules/
        {store_id}-{roothash}.dig # compiled module (WASM binary), one per generation
      config.toml             # store configuration (0600 if private)
      signing_key.bin         # BLS signing-key seed (0600; never embedded)
      trusted_keys.json       # trusted host keys (public)

```

The compiled module is the distribution unit. Backups copy the module; verification opens it; serving instantiates it. Each store under `stores/<name>/` keeps its own staging area, generations, compiled modules, and keys, so stores in one workspace are fully isolated.

Multiple stores per project

A workspace tracks any number of named stores. `workspace.toml` records the **active** store, a name → store-id registry, and each store's optional content root:

```

active = "default"

[stores.default]
id = "ab12...64hex..."

[stores.site]
id = "cd34...64hex..."
content_root = "dist"

```

Store names are local, human-friendly aliases (letters, digits, `.`, `_`, `-`); the 64-hex store ID (§4.2) remains the cryptographic identity that URNs use. The target store for a command is resolved by precedence: an explicit `--store <name>` flag wins; otherwise the workspace's active store; otherwise, if exactly one store exists, that single store; otherwise the command errors and asks you to select one. `digstore stores` lists the stores, `digstore use <name>` sets the active store, and `digstore init <name>` adds a new one. `clone` materializes its store as `default`.

Content root — what `add / urn` capture

Unlike Git, digstore captures **build output**, not repository files, so the directory a store captures is a first-class, configurable notion distinct from where `.dig` lives. Each store has a **content root**: the directory that `add` and `urn` scan, and the root that resource keys are relative to. It defaults to the working directory and is commonly set to a build directory (e.g. `dist/`) via `digstore init <name> --dir dist` or `digstore dir <path>`. Because keys are content-root-relative, a file at `<project>/dist/css/app.css` always gets the key `css/app.css` and a stable URN, regardless of which subdirectory a command runs from. The global `-C/--cwd <path>` flag overrides the content root for a single command.

5. The Store Module

The store module is a WebAssembly binary produced by the compiler (`digstore-compiler`). It is the single file a host serves. **Compiler version 1.0.0; module format version 1.**

5.1 Module Sections

Section	Contents
Type	Function signatures (<code>[] → [i64]</code> , <code>[i32, i32] → [i64]</code> , etc.)
Import	Host functions in the <code>dig_host</code> module (§6.3)
Function	Type indices for each defined function
Memory	One linear memory: min 1 page (64 KiB), max 6144 pages (384 MiB)
Export	The store ABI (§6.2) plus <code>memory</code> , <code>alloc</code> , <code>dealloc</code> , <code>init</code>
Code	Function bodies, obfuscated if enabled (§17)
Data	Embedded content: chunks, key table, metadata, trusted keys (no secret)


```
MemoryType {
  minimum: 1,           // 64 KiB
  maximum: Some(6144),  // 384 MiB module-declared cap
  memory64: false,
  shared: false,
}
```

The module-declared memory cap (384 MiB) is the inner bound. It is sized to hold the embedded data section — up to the per-store **128 MB content cap** plus AEAD/merkle/filler overhead — together with the working heap a serve needs. Crucially, the guest heap is **not** at a fixed offset: the bump allocator places its base **dynamically above** the data section at `align_up(DIGS_DATA_OFFSET + blob_len, 64 KiB)`, computed from the embedded `DIGS` header on the first allocation. So heap growth can never overlap the embedded chunk pool for **any** blob size — contract D2 holds whether the data section is a few kilobytes or the full cap. Serving is **single-copy**: a worst- case single resource of ~122 MB materializes once and serves comfortably within the 384 MiB ceiling. The host runtime additionally enforces an outer memory limit, table and instance limits, a fuel budget, and a wall-clock timeout (§18.2).

5.2 What Gets Embedded

The compiler loads every generation from disk, deduplicates chunks across generations into a single chunk index, builds a resource-key table, and embeds the result in the data section. Embedded, for each generation:

- **Chunks (the interleaved pool).** The deduplicated, encrypted chunk bodies, each addressed by `SHA-256` of its ciphertext, placed into one interleaved pool with deterministic filler in the gaps and no resource boundaries (§8.3). A chunk shared across generations is stored once.
- **Key table.** Maps each resource key to the ordered chunk indices and total size that reassemble it.
- **Metadata.** Store ID, root history, store public key, authentication info.
- **Metadata manifest.** A plaintext, publisher-authored manifest of descriptive store and authorship information (§8.4). Unlike content, the manifest is not encrypted, so any holder of the module can read it without a URN.
- **Trusted host keys.** The set of BLS host public keys the module attests against (§12).

The module embeds **no secret**: no signing key and no decryption key. Content keys live with URN-holders (§11); execution proofs use no secret (§13). A dump of the data section is ciphertext, public metadata, and public keys.

Data-section format (binding). The data section is a length-prefixed blob beginning with the ASCII magic `DIGS`, a one-byte format version (`1`), a `u32` section count, and an offset table of `(id:u16, offset:u32, len:u32)` rows in ascending section-id order, followed by the section bodies. The eleven sections (ids 1–11) are: `StoreId`, `CurrentRoot`, `RootHistory`, `PublicKey`, `TrustedKeys`, `Metadata`, `AuthInfo`, `KeyTable`, `ChunkPool`, `MerkleNodes`, `Filler`. **All multi-byte integers are big-endian** (Chia "streamable" framing; this is a deliberate deviation from a little-endian note in earlier drafts — the implementation, guest reader, and client verifier all agree on big-endian). The

encoding is canonical: sections appear exactly once, in id order, and decoders that round-trip the blob reproduce identical bytes, which is what lets the program hash be a stable identifier (§19.3).

5.3 Compilation Pipeline

The compiler runs a fixed sequence of stages (`digstore-compiler`):

1. **Config load.** Parse store configuration, extract `store_id` , validate required fields.
2. **Generation load.** Read each `generations/{roothash}/` directory: manifest, chunk files; build the per-generation merkle tree.
3. **Chunk process.** Deduplicate chunks across all generations into the global chunk index.
4. **Key-table build.** Map resource keys to chunk-index sequences per generation.
5. **Code build.** Emit the type, import, function, memory, and export sections.
6. **Data embed.** Encode chunks, metadata, key table, and trusted keys into the data section (big-endian binary codec).
7. **Obfuscation.** Optionally apply code-obfuscation passes (§17).
8. **Optimization.** Optional `wasm-opt` .
9. **Validation.** Re-parse and validate the emitted module.
10. **Output.** Write atomically to `{hex(store_id)}-{hex(roothash)}.dig` via a temporary file and rename.

Compilation requires at least one trusted host key; the compiler refuses to emit a module with an empty trusted-key set (`CompilerError::NoTrustedKeys`).

6. The Host/Module ABI

The host and module communicate across a fixed application binary interface. All data crosses the boundary through linear memory. Integer return values pack a pointer and a length.

6.1 Return Encoding

```
pub const fn pack_ptr_len(ptr: u32, len: u32) → i64 {
    ((ptr as i64) << 32) | (len as i64)
}
pub const fn unpack_ptr_len(packed: i64) → (u32, u32) {
    ((packed >> 32) as u32, (packed & 0xFFFF_FFFF) as u32)
}
// Error sentinel: len == 0 and (ptr as i32) < 0 ⇒ ptr is an error code.
pub const fn is_error(packed: i64) → bool {
    let (ptr, len) = unpack_ptr_len(packed);
    len == 0 && (ptr as i32) < 0
}
```

6.2 Module Exports

Export	Signature	Returns
<code>get_store_id</code>	<code>() → i64</code>	Store ID (32 bytes)
<code>get_current_roothash</code>	<code>() → i64</code>	Current generation root hash
<code>get_roothash_history</code>	<code>() → i64</code>	Array of all root hashes
<code>get_public_key</code>	<code>() → i64</code>	Store public key (48 bytes, BLS G1)
<code>get_metadata</code>	<code>() → i64</code>	Metadata manifest, plaintext (§8.4)
<code>get_authentication_info</code>	<code>() → i64</code>	<code>AuthenticationInfo</code>
<code>get_content</code>	<code>(req_ptr: i32, req_len: i32) → i64</code>	<code>ContentResponse</code> or decoy
<code>get_proof</code>	<code>(req_ptr: i32, req_len: i32) → i64</code>	<code>ProofResponse</code>
<code>alloc</code>	<code>(size: i32) → i32</code>	Pointer into linear memory
<code>dealloc</code>	<code>(ptr: i32, size: i32) → ()</code>	(none)
<code>init</code>	<code>() → i32</code>	0 = success, < 0 = error
<code>memory</code>	(exported memory)	Linear memory

`get_content` and `get_proof` take a request encoded into module memory by the host: the host calls `alloc`, writes the request, then calls the export with the pointer and length. `get_metadata` takes no request and returns the plaintext manifest directly; it is not gated by the URN, the session, or attestation (§8.4).

6.3 Host Imports (`dig_host`)

The module imports host services. Each import returns an `i32`: a non-negative value is the length written to the shared return buffer; a negative value is an error code. The module reads results back with `host_read_return_buffer`.

Import	Signature	Purpose
<code>host_get_public_key</code>	<code>() → i32</code>	Host's BLS public key
<code>host_create_attestation</code>	<code>(challenge_ptr) → i32</code>	Host signs an attestation challenge
<code>host_establish_session</code>	<code>(challenge_ptr) → i32</code>	Create a session after attestation
<code>host_verify_session</code>	<code>() → i32</code>	1 = valid, 0 = invalid
<code>jwt_fetch</code>	<code>(url_ptr, url_len) → i32</code>	Fetch a JWKS document for JWT validation
<code>host_get_current_time</code>	<code>() → i64</code>	Unix timestamp (seconds)
<code>host_random_bytes</code>	<code>(count) → i32</code>	Cryptographic random bytes
<code>host_read_return_buffer</code>	<code>(dest_ptr) → i32</code>	Copy the host return buffer into module memory

`jwt_fetch` is session-gated: the host returns `NoSession` (-100) or `SessionExpired` (-101) until the module has established a valid session. Identity and attestation imports bootstrap the session and do not require one.

`jwt_fetch` is **SSRF-guarded**. Because the guest fully controls the URL, the host requires `https` and refuses any URL that resolves to a loopback, private (RFC 1918), link-local (including the `169.254.169.254` cloud-metadata address), CGNAT, or IPv6 unique-local/link-local address. A development override exists but is insecure by name and must not be set on a public host.

JWT authorization — current scope. A JWT gate is an *optional, per-store* policy. In the current implementation the gate parses the token and checks claims (issuer, audience, expiry) but **does not yet verify the token signature against the fetched JWKS**, and the gate is disabled by default. JWT authorization must therefore not be relied upon as a security boundary until signature verification is wired (see §22, residual risk R4). The building blocks — `jwt_fetch`, JWKS parsing, RS256/ES256 verification — exist; what remains is to enforce them on the content path under a deterministic, proof-compatible design.

6.4 The Return Buffer

The host owns a shared return buffer; import results are written into it and the module reads out of it.

```
pub struct HostImportsConfig {
    pub return_buffer_capacity: usize, // default 64 KiB
    pub max_return_buffer_size: usize, // default 16 MiB
    pub max_random_bytes: u32,        // default 1024
    pub host_version: String,
}
```

6.5 Error Codes

```
pub enum ErrorCode {
    GeneralError      = -1,
    InvalidParameter  = -2,
    BufferTooSmall     = -3,
    NoSession         = -100,
    SessionExpired    = -101,
    AttestationFailed = -102,
    NetworkError      = -200,
    Timeout           = -203,
    NotFound          = -300,
    ValidationFailed  = -301,
}
```

7. URN System

Every resource is addressed by a Uniform Resource Name. The URN is the sole input to both locating and decrypting a resource (§1, §11). An invalid URN is indistinguishable from a valid one at retrieval time (§14).

7.1 Format

```
urn:dig:<chain>:<storeID>[:<rootHash>][/<resourceKey>]
```

```
pub struct Urn {
    pub chain: String,           // chain identifier, e.g. "chia"
    pub store_id: Bytes32,       // 32-byte store id (required)
    pub root_hash: Option<Bytes32>, // optional generation pin
    pub resource_key: Option<String>, // optional resource path
}
```

7.2 Components

Component	Role	Required
urn	Scheme	Required. The literal <code>urn</code> .
dig	Namespace	Required. The literal <code>dig</code> .
<chain>	Chain	Required. Chain identifier (e.g. <code>chia</code>).
<storeID>	Store ID	Required. 64 hex characters.
<rootHash>	Generation	Optional. Pins a generation; absent means the current root.
<resourceKey>	Resource	Optional. Path-like key; absent means store-level.

7.3 Resolution

The URN is canonicalized, then two values are derived from the canonical string and nothing else:

- **Retrieval key.** `retrieval_key = SHA-256(canonical_urn)` . The 32-byte key under which the resource is located. This is the only address that ever leaves the client; the URN itself does not.
- **Decryption key.** `decryption_key = HKDF-SHA256(ikm = canonical_urn, ...)` . The AES-256 key used to decrypt the resource's chunks (§11).

If `rootHash` is omitted, resolution targets the current generation; if present, that exact generation. A resolved `resourceKey` maps through the module's key table to the ordered chunk indices that reassemble the resource. Resolution that finds no entry does not fail: it returns a deterministic decoy (§14).

Note on rootless URNs. A URN without a `rootHash` derives a *root-independent* key, so the same key encrypts the resource across generations. This is why the chunk cipher must be nonce-misuse resistant (§11.2): the same key recurs over distinct plaintexts.

8. Content Model

8.1 Chunking

Content is split into variable-size chunks by content-defined chunking. A Gear-based rolling hash slides over the byte stream; a boundary is cut where the masked rolling hash matches, bounded by a minimum and a maximum size. Identical content produces identical boundaries regardless of surrounding edits, which maximizes deduplication across generations.

```
pub struct ChunkerConfig {
    pub min_size: usize,      // 16 KiB
    pub target_size: usize,   // 64 KiB
    pub max_size: usize,      // 256 KiB
    pub mask: u64,            // boundary mask for the target size
}
```

Parameter	Value	Effect
Minimum chunk	16 KiB	Floor on chunk size; suppresses pathological tiny chunks
Target chunk	64 KiB	Expected average; sets the boundary mask width
Maximum chunk	256 KiB	Hard cap; forces a boundary if none is found

Each chunk is hashed with SHA-256. After encryption, the **ciphertext** is what is content-addressed and committed (§9.1), so the stored address is `SHA-256(ciphertext)` .

8.2 Generations

A generation is the complete chunk set for one store state. A commit deduplicates the new state's chunks against the existing index, builds the generation's merkle tree over its per-resource leaves, and records the

root. Chunks shared with prior generations are not re-stored. The generation's root hash is its identifier and is appended to the root history.

8.3 The Interleaved Chunk Pool

Inside the module, chunks are not grouped by resource. Every chunk across every resource is placed into a single interleaved pool, the gaps padded with deterministic filler, and no resource boundary is observable in the data section. A resource is reassembled by a path walk: the key table yields the ordered chunk indices, and the module gathers exactly those chunks from the pool in order.

```
pub struct PathWalk {  
    pub resource_key: Bytes32,    // requested resource  
    pub chunk_indices: Vec<u32>, // ordered indices into the pool  
    pub cursor: usize,           // current position in the walk  
}
```

Because the pool carries filler and no boundaries, the data section reveals neither how many resources a store holds nor where one ends and the next begins. Reassembly requires the key-table entry, which requires the resource key, which requires the URN.

Uniform module size. The filler does more than hide boundaries: every module's injected data blob is padded to **one fixed size** (a budget covering a full 128 MB store's ciphertext, key table, merkle nodes, and headers), so every store compiles to the **same module size** regardless of how much content it holds. A store at 100% of the cap carries ~no filler; a smaller store is padded with deterministic filler up to the fixed budget. As a result the module size reveals nothing about content size — there is no coarse size bucket to read off, and two stores with very different amounts of content are byte-for-byte the same length. The filler is the deterministic keystream of §19.3, so the padded module stays reproducible.

8.4 The Metadata Manifest

A store carries a publisher-authored metadata manifest: a structured, plaintext description of the store, its authorship, and related information, in the spirit of a package manifest. It is embedded in the data section and read through the `get_metadata` export (§6.2). It is the one part of a store that is deliberately readable without a URN.

Plaintext by design. Unlike content, the manifest is not encrypted (§11). It is the store's public face: a host, indexer, or browser can read it directly from the module, with no URN, session, or attestation. Content remains URN-gated and provider-blind; the manifest is intentionally open so a store can describe itself. A publisher who wants no public description ships an empty manifest.

The manifest is a set of standard fields (`schema_version` , `name` [required] , `version` , `description` , `authors` , `license` , `homepage` , `repository` , `keywords` , `categories` , `icon` , `content_type` , `links`) plus an open-ended `custom` map.

```
pub struct MetadataManifest {
    pub schema_version: u32,           // required
    pub name: String,                 // required
    pub version: Option<String>,
    pub description: Option<String>,
    pub authors: Vec<Author>,
    pub license: Option<String>,
    pub homepage: Option<String>,
    pub repository: Option<String>,
    pub keywords: Vec<String>,
    pub categories: Vec<String>,
    pub icon: Option<String>,
    pub content_type: Option<String>,
    pub links: BTreeMap<String, String>,
    pub custom: BTreeMap<String, Value>, // open-ended
}
```

Editing and re-compilation. The manifest is part of the module bytes, so changing it changes the module and therefore the program hash. A metadata edit is a re-compilation, not a new generation: the content root is unchanged (no content chunk changed) while the program hash advances (the bytes did).

8.5 Social Conventions

Content addressing is private by default: without the exact URN, a resource cannot be located or read. Social conventions are an optional, opt-in layer that makes selected resources discoverable without weakening the guarantee for everything else.

A resource key is the only secret part of a URN; the store ID and chain are public, and the URN derivation is public. So a party that already knows a store's ID can construct the URN for any agreed, well-known resource key, derive its retrieval key, fetch it, and — on a public store — derive its decryption key and read it.

Convention	Resource key	Purpose
Default resource	<code>index.html</code>	The store's landing resource, rendered as its default view
Discovery manifest	<code>/.well-known/dig/manifest.json</code>	A machine-readable list of the resources the publisher chooses to expose

On a public store, a conventional resource is both locatable and readable by anyone who knows the store ID. On a private store, the conventional resource key still yields a retrieval key, but decryption additionally requires the secret salt (§11.4), so the resource is locatable in principle yet sealed in practice. Conventions are conventions, not protocol requirements: a consumer lists only what the publisher placed at conventional keys and cannot enumerate secret-keyed content.

9. Merkle Proofs

Every response can be bound to a generation's root by a merkle inclusion proof, so a client can verify that the bytes it received belong to the store state it trusts.

9.1 Tree Construction

A generation's **leaves are per-resource ciphertext digests**, not per-chunk hashes. For each resource, the leaf is `SHA-256(concat(ordered ciphertext chunks of that resource))` — i.e. SHA-256 of exactly the bytes `get_content` returns for that resource. Leaves are ordered ascending by the resource's static retrieval key. An interior node is `SHA-256(left || right)`; when a level has an odd number of nodes, the last node is carried up. The root is the generation's root hash.

This commits the served ciphertext directly, so a verifier checking a returned resource recomputes one leaf and folds the path to the trusted root. (Earlier drafts described a per-chunk leaf of `SHA-256(chunk)`; the implementation commits per-resource ciphertext, as above.)

9.2 Inclusion Proof

```
pub struct MerkleProof {
    pub leaf: Bytes32,           // SHA-256 of the resource's ciphertext blob
    pub path: Vec<ProofStep>,    // sibling hashes from leaf to root
    pub root: Bytes32,           // claimed generation root
}
pub struct ProofStep {
    pub hash: Bytes32,           // sibling hash at this level
    pub is_left: bool,           // sibling's side
}
```

9.3 Verification

The verifier starts from the leaf and folds each step: combine the running hash with the sibling on the indicated side, hash, and ascend. The proof is accepted if and only if the recomputed root equals the trusted root.

9.4 Binding to the Trusted Root

The verifier supplies the root from the store's root history, or from an external anchor it trusts — never from the response. Any altered, substituted, or truncated content changes its leaf, breaks the path, and fails to recompute the trusted root. A host cannot fabricate content that verifies, and the module holds no secret that would help it try. Integrity rests on the hash function and the trusted root alone. (How a remote client *learns* a trustworthy root in the first place is the authenticated-head mechanism of §21.6.)

9.5 Proof Size

A proof carries $\lceil \log_2(n) \rceil$ sibling hashes for a generation of n resources, 32 bytes each. Verification is logarithmic and requires only the leaf, the path, and the trusted root.

Part 3: Security and Zero-Knowledge

10. Threat Model

10.1 Adversaries

- **A curious or malicious provider.** A DIG Node that stores and serves the module and wishes to learn what it carries or to alter what it returns.
- **A network observer.** A party between client and provider watching requests and responses.
- **An enumerating client.** A party without the URN attempting to discover which addresses map to real content.
- **A malicious remote/origin.** A remote endpoint that serves clones/pulls and may attempt to substitute a fabricated module or root.

10.2 Guarantees

- **Confidentiality.** Content is unreadable without the URN, because the decryption key is derived from the URN (§11).
- **Integrity.** Responses are bound to the trusted root by a merkle proof (§9) and to the URN by AEAD authentication (§11), and — on a remote — the served root is bound to the publisher's BLS signature (§21.6).
- **Blindness.** The provider receives a retrieval key (a hash) and returns ciphertext. It cannot scan a store at rest or inspect a request in flight (§15).
- **Indistinguishability.** Invalid URNs return deterministic decoys that look like real responses (§14).

10.3 Assumptions and Non-Goals

- The primitives are secure: SHA-256, AES-256-GCM-SIV, HKDF-SHA256, BLS signatures, and (when enabled) the zero-knowledge proof system.
- The URN is a capability: anyone who learns it gains access. How URNs are distributed and to whom is out of scope. The format defends content against a hostile provider and a network observer; it does not defend a URN that its holder discloses.
- Resistance to traffic analysis beyond decoys and oblivious access (§14) is out of scope.
- **Availability is not a primary goal.** The host runtime and remote enforce resource bounds (memory, table, fuel, wall-clock, request body size) and the remote applies per-store rate limiting, but a determined adversary with sufficient resources is not in scope for a denial-of-service guarantee.

11. URN-Based Encryption

11.1 Key Derivation

The decryption key for a resource is derived from its canonical URN by HKDF-SHA256, yielding a 32-byte AES-256 key:

```
salt = SHA-256("digstore-hkdf-salt-v1" [|| secret_salt]) # secret_salt only for private store
decryption_key = HKDF-SHA256(ikm = canonical_urn, salt, info = "digstore-aes-256-gcm-key-v1")
```

The key is a deterministic function of the URN (and, for a private store, the salt). No key material is stored in the module, transmitted, or held by the provider. Possession of the URN is necessary and sufficient to derive the key for a public store.

11.2 Encryption

Each resource's chunks are encrypted with **AES-256-GCM-SIV** (RFC 8452), a nonce-misuse-resistant authenticated cipher, under the resource's URN-derived key. A fixed all-zero nonce is used.

GCM-SIV derives a synthetic IV internally (POLYVAL over the key and plaintext), so each distinct plaintext is sealed under an independent effective IV even when the public nonce is held constant. This is what keeps a fixed nonce safe — and it **must** be GCM-SIV, not plain GCM. A rootless URN (one that targets the current generation, §7.3) reuses its derived key to encrypt a *different* plaintext in every generation, so the same key and fixed nonce recur across distinct plaintexts. Under plain GCM that reuse leaks the keystream XOR and permits recovery of the GHASH authentication key (the "forbidden attack"); GCM-SIV is specifically designed to tolerate it. The only thing a fixed nonce can leak under GCM-SIV is whether two (plaintext, key) pairs are identical — which content-addressed deduplication already reveals — so no confidentiality or integrity guarantee is weakened.

Holding the nonce fixed also keeps encryption **deterministic**, which the ciphertext-committed merkle root (§9) requires: the same plaintext under the same key must always seal to the same ciphertext, or the committed root would not be reproducible. The authentication tag binds the ciphertext to the key; tampering is detected on decryption and surfaced as a failure rather than silently accepted.

11.3 Decryption

Decryption runs on the client, after it receives the ciphertext chunk. The client derives the key from the URN it holds, verifies the tag, and recovers the plaintext. The provider performs no decryption and never holds the key. A failed tag is reported as tampering.

11.4 Private Stores

A private store mixes a secret salt into the key derivation, so the URN alone is insufficient to derive the decryption key. The salt is held by the publisher and shared out-of-band with authorized readers. A URN-

holder who lacks the salt can locate a resource (the retrieval key still derives from the URN) but cannot read it. This is the mechanism behind the public-versus-private distinction in the visibility configuration (§4.1) and in social-convention discovery (§8.5).

12. Host Attestation and Sessions

A store module refuses to serve content until the host proves its identity. The module embeds a set of trusted BLS keys at compile time (§5.2), and attestation binds the running host to one of those keys. A host that cannot attest receives decoys, never content.

12.1 The Handshake

```
pub struct AttestationChallenge {
    pub nonce: [u8; 32],      // module-generated random nonce
    pub store_id: [u8; 32],    // the store being served
    pub timestamp: u64,        // unix seconds, for freshness
}

pub struct AttestationResponse {
    pub host_public_key: [u8; 48], // host BLS public key (G1)
    pub host_instance_id: [u8; 32], // host instance identifier
    pub signature: [u8; 96],        // BLS signature (G2) over the challenge
}
```

The module calls `host_create_attestation` with the challenge. The host signs it with its BLS key and returns the response through the return buffer.

12.2 Verification

The module verifies the BLS signature over the challenge under `host_public_key`, checks the timestamp for freshness, and checks that `host_public_key` is a member of the trusted set embedded at compile time. If any check fails the module refuses (`AttestationFailed`, -102) and subsequent content calls return decoys. There is no fallback path that serves content without attestation.

12.3 BLS Signing Roles

Digstore uses BLS signatures (Chia AugScheme over BLS12-381) for three distinct roles: push authorization (`SHA-256(root || store_id)`, §21.6), host attestation (the challenge `nonce || store_id || timestamp`, §12.1), and node attribution on execution proofs (`proof || public_input`, §13.7). These messages are presently distinguished only structurally — by differing fixed lengths and by binding the `store_id` into both the push and attestation messages — rather than by an explicit domain tag. Adding a per-role domain-separation prefix (`digstore-push-v1 / digstore-attest-v1 / digstore-node-v1`) is a recommended defense-in-depth hardening that forecloses any cross-role reuse argument; it is a breaking signature change tracked as a residual item (§22, R2).

12.4 Trusted Key Format and Sessions

Trusted host keys are recorded as versioned entries `dig-host-key-v1:<hex>`, where the hex encodes a BLS public key; the compiler embeds one or more and refuses an empty set (§5.3).

After successful attestation the module establishes a session through `host_establish_session`. Session-gated imports such as `jwt_fetch` succeed only while a valid session exists; before that they return `NoSession` (-100), and an expired session returns `SessionExpired` (-101). The session is the precondition for any JWT-authorization logic the module enforces before releasing real content (§6.3).

13. Execution Proofs

Beyond proving content genuine (§9), a serving node can prove the serving *computation* correct. An execution proof is a succinct, zero-knowledge attestation about a deterministic re-execution of the serving path.

13.1 What Is Proven (and What Is Not)

A proof attests the statement: re-running the **deterministic serving computation** on the resolved request — key-table lookup, in-order gather of the resource's chunk ciphertext, and the output commitment `public_output = SHA-256(roothash || concat(ciphertext))` — reproduces the committed output. `program_hash = SHA-256(module bytes)` is carried in the proof to **identify the served build**.

Scope, stated precisely. In the current proving circuit, `program_hash` is a bound public value of the statement, not a constraint that the prover actually executed *that* WASM: the circuit re-runs the serving computation, not wasmtime opcodes. Proving opcode-level WASM execution against the program hash is future work. The proof therefore attests "this output is the correct deterministic serving result for these inputs," and the program hash is an accompanying identifier rather than a proven binding. Content integrity does not rest on the proof — it rests on the merkle root (§9) and the authenticated head (§21.6), which are in force regardless of proof mode.

Deployment status. The default proof backend is a *mock* prover for development, which produces a recomputable digest rather than a sound zero-knowledge proof, paired with a fixed clock and a mock chain source. The RISCo backend, a real chain source, and a real clock must be enabled before execution proofs carry cryptographic weight (see §22, residual risk R3).

13.2 Proof Structure

```
pub struct ExecutionProof {
    pub program_hash: Bytes32, // SHA-256 of the served module (identifier)
    pub public_input: Vec<u8>, // request binding: client nonce + recent Chia block (§13.8)
    pub public_output: Bytes32, // commitment to the output bytes
    pub proof: Vec<u8>, // succinct zero-knowledge proof
    pub chia_block: ChiaBlockRef, // recent Chia block header; bound into public_input
    pub node_pubkey: [u8; 48], // serving node BLS public key (G1)
    pub node_signature: [u8; 96], // BLS signature (G2) over (tag || proof || public_input)
}

pub struct ChiaBlockRef {
    pub header_hash: Bytes32, // header hash of a recent Chia block
    pub height: u32, // block height
    pub timestamp: u64, // block timestamp (unix seconds)
}

pub struct ProofResponse {
    pub proof: ExecutionProof,
    pub roothash: Bytes32, // generation the response is bound to
}
```

13.3–13.5 Pipeline, Verification, Nonce Binding

The serving node executes the serving computation inside the proving environment, which emits the proof together with the public input and the output commitment. A verifier checks the proof against `program_hash` and the public input, confirms the output commitment matches the bytes received, and confirms the response is bound to a root it trusts. The public input includes a fresh client nonce, so a proof is valid only for the request that produced it: a node cannot replay an old proof and a relay cannot detach a proof from its request.

13.6 Cost and the Hardware Alternative

Generating a zero-knowledge proof per request is the most expensive operation in the serving path. Where that cost is prohibitive, the format permits execution inside a trusted execution environment or HSM-attested enclave that vouches for the run, trading the cryptographic proof for a hardware attestation of the same statement. Content retrieval and merkle verification are unaffected.

13.7 Node Attribution

The serving node is identified by the BLS key it uses for attestation (§12), one key for both roles.

`node_signature` is a BLS signature over the node-attribution message (`proof || public_input`), so the run is attributed to that node and the attribution is verified under `node_pubkey` . The client-nonce binding keeps the attribution from being lifted onto another request. (A per-role domain tag on this message is a recommended hardening — §12.3, §22 R2.)

13.8 Chain-Anchored Freshness

A proof also commits to a recent Chia block header, which bounds when the proof could have been produced. A Chia block header hash cannot be known before the block is produced, so a valid proof committing to a given header could only have been generated after that block existed. A verifier reads the committed timestamp and height, confirms the header is a real block on the chain it trusts, and rejects the proof if the block falls outside an acceptable freshness window. The nonce defeats replay; the block header defeats precomputation.

14. Oblivious Retrieval

Two mechanisms keep a provider from learning what a client requested: decoys make a miss look like a hit, and an oblivious access pattern decorrelates in-module pool access from the resource.

14.1 The Indistinguishability Goal

A provider that serves requests must not be able to separate real lookups from misses, nor trivially infer which resource a real lookup concerned, so that enumeration yields no signal and traffic patterns leak little about content.

14.2 Decoys

A retrieval key that resolves to no entry does not produce an error. The module returns deterministic decoy bytes whose size is drawn from a logarithmic distribution seeded by the requested key, under a normal success status. A miss returns a 200, never a 404. Because the decoy is a deterministic function of the key, the same miss always returns the same bytes, so a decoy is indistinguishable from a genuine immutable response and is as cacheable as any other.

14.3 Oblivious Access

When a real lookup is served, the module gathers the resource's chunks from the interleaved pool (§8.3) through an access pattern designed to be decorrelated from the request: pool accesses are padded and reordered so the observable sequence does not directly reveal which chunks, or how many, belong to the requested resource. Combined with the absence of resource boundaries in the pool, this denies the provider both the content and the obvious shape of what it serves. The strength of this obliviousness against a sophisticated statistical adversary is a function of the padding/reordering policy and is an area of ongoing hardening (§22, R5); the decoy and provider-blindness guarantees do not depend on it.

15. Provider Blindness and the Neutral Pipe

Provider blindness is the consequence the whole format is arranged to produce. It is structural, not a matter of provider policy.

A provider holds the module — code plus an interleaved pool of ciphertext and filler plus public metadata and public keys — and receives requests addressed by a retrieval key (a 32-byte hash). It returns ciphertext and proofs. It holds no URN and no decryption key.

The provider can observe	The provider cannot learn
That a module is a store (its fixed size, uniform across stores, §8.3)	The plaintext of any content, or how much content the store holds
Retrieval keys requested (hashes)	The URN behind a retrieval key
That a request occurred and its timing	Which resource a request concerned, or whether it hit
The public metadata manifest (§8.4)	Anything the publisher did not place in the manifest

The retrieval key is a hash of the URN, so a provider cannot reverse it to recover the URN, and without the URN it can derive no decryption key. Neutral-pipe status does not depend on the provider behaving well: the keys it would need to read content are functions of a URN it never receives. The guarantee is cryptographic, not contractual.

16. Temporal Keys

Access can be bounded in time. A capability may carry a validity window, and the module enforces it using the host clock (`host_get_current_time` , §6.3). A capability outside its window is treated like any other unauthorized request: the module returns a decoy, not an error, preserving indistinguishability (§14). Because the check uses the host-supplied time, it depends on the host clock the attestation handshake established trust in (§12).

17. The Secretless Module

The module embeds no secret of any kind. This is the property that makes wide replication safe and provider blindness structural.

17.1 Obfuscation

The compiler can optionally apply deterministic code obfuscation to the serving logic: instruction substitution, opaque predicates, bogus code paths, and control-flow nops. Obfuscation raises the cost of reverse-engineering the module's logic, but the format's security does not rest on it. A fully de-obfuscated module still reveals only ciphertext, public metadata, and public keys.

17.2 No Embedded Secret

- **No decryption key.** Content keys are derived from URNs and held by readers (§11), never stored in the module.

- **No signing key.** A store's signing identity is the publisher's BLS key, used out-of-band; the module does not carry it.
- **No proving secret.** Execution proofs are verified against the public program hash and use no embedded secret (§13).

Because there is nothing to extract, copying the module is safe: a stolen module is exactly as useful as a public one — it serves only ciphertext to anyone lacking the URNs. Security lives in the URNs, not in the artifact.

Note on client-side key hygiene. The *client* does hold URNs and derived keys in memory while decrypting. Derived key material is not yet explicitly zeroized after use (best-effort scrubbing is a recommended hygiene item; Rust offers no hard guarantee regardless). The primary at-rest exposure is the persisted key files (`signing_key.bin` , the salt), which are protected by owner-only file permissions (§4.1).

Part 4: Runtime and Tooling

18. The Host Runtime

The host runtime (`digstore-host`) loads a module, supplies the `dig_host` imports, and executes exports inside a sandbox with hard bounds. It never parses content out of the module; it interacts only across the ABI (§6).

18.1 Instantiation

The runtime parses and validates the module, instantiates it on a wasmtime-class engine, and wires the host imports. The data section is opaque to the runtime: it is reached only by calling exports, which return packed pointer/length results into linear memory. The WebAssembly threads/shared-memory proposal is disabled — a serve-only module has no use for it, and disabling it keeps all guest memory inside the runtime's accounting.

18.2 Execution Bounds

- **Wall-clock timeout.** Each export call is bounded in time via epoch interruption; an overrun traps.
- **Fuel metering.** A fuel budget bounds work per call.
- **Memory ceiling.** The runtime enforces an outer linear-memory limit that defaults to **384 MiB** — matching the module's declared cap (§5.1) — and is operator-configurable via `ExecutionLimits.memory_bytes_max` . This outer bound is the real DoS limit for an untrusted module.
- **Table and instance limits.** Table element count, table count, memory count, and instance count are all bounded, so a guest cannot exhaust host memory via `table.grow` or excess instantiation.

- **Untrusted-module size bound.** A module fetched from an untrusted source is size-checked before validation/compilation, so an oversized blob cannot exhaust memory during instantiation.

18.3 The Import Surface

The runtime implements `host_get_public_key`, `host_create_attestation`, `host_establish_session`, `host_verify_session`, `jwt_fetch` (SSRF-guarded, §6.3), `host_get_current_time`, `host_random_bytes` (sourced from the OS CSPRNG), and `host_read_return_buffer`. Results are written into the shared return buffer (§6.4).

18.4 Serving a Request

1. host calls `alloc(request_len) → ptr`
2. host writes the encoded request at `ptr`
3. host calls `get_content(ptr, request_len) → packed i64`
4. if `is_error(packed)`: handle error code
else: `(out_ptr, out_len) = unpack_ptr_len(packed)`
5. host reads `out_len` bytes at `out_ptr` (ciphertext + proof, or decoy)
6. host calls `dealloc` as needed

The same pattern serves `get_proof`. The runtime returns to the client exactly what the module produced: it neither decrypts nor inspects the payload.

19. Compilation

Compilation is the deterministic transform from on-disk generations to a single module file (§5.3).

19.1 Inputs

- The store configuration and store ID.
- Every generation directory under `generations/`.
- The set of trusted host keys to embed (§12.4).
- Compiler options: obfuscation, optimization.

19.2 Trusted Host Keys

```
pub struct TrustedHostKey {  
    pub public_key: [u8; 48], // host BLS public key (G1)  
    pub label: String,        // versioned label, e.g. "dig-host-key-v1:<hex>"  
}
```

At least one trusted host key is required; the compiler refuses an empty set.

19.3 Determinism

Compilation is deterministic: the same inputs produce byte-identical output. The data-section codec is canonical (§5.2), the chunk cipher is deterministic (§11.2), and the filler is a deterministic keystream. This is what lets the program hash function as a stable identifier and lets a verifier confirm an execution proof against it (§13). Two parties compiling the same store with the same trusted keys obtain the same module and the same program hash.

20. Developer Experience: A Git-Compatible Workflow

The CLI (`digstore`) presents Git's verbs and mental model.

20.1 Store identity at `init`

`digstore init` generates a BLS keypair, sets `store_id = SHA-256(public_key)` , records the visibility choice, and writes the local configuration and directory layout. The signing-key seed is persisted with owner-only permissions and is never embedded in a module. All key/salt material is drawn from the OS CSPRNG.

20.2 Command Reference

Command	Form	Effect
<code>init</code>	<code>digstore init</code>	Create a new store and local layout
<code>add</code>	<code>digstore add <path></code>	Stage and chunk content
<code>commit</code>	<code>digstore commit</code>	Create a generation and compile the module
<code>status</code>	<code>digstore status</code>	Show staged changes
<code>log</code>	<code>digstore log</code>	List generations (root = commit id)
<code>diff</code>	<code>digstore diff <a> </code>	Compare two generations
<code>checkout</code>	<code>digstore checkout <root></code>	Materialize a generation
<code>cat</code>	<code>digstore cat <urn></code>	Read a resource by URN
<code>remote</code>	<code>digstore remote add <name> <url></code>	Configure a remote
<code>clone</code>	<code>digstore clone <urn\ url></code>	Fetch and verify a store from a remote
<code>push</code>	<code>digstore push <remote></code>	Push to a remote (§21)
<code>pull</code>	<code>digstore pull <remote></code>	Pull from a remote (§21)

checkout path safety. Resource keys come from a (possibly cloned, untrusted) store's key table. `checkout` rejects any key that would escape the output directory (`..` , absolute paths, Windows drive

prefixes or alternate data streams), so a malicious store cannot write files outside the target.

Remote URL policy. `clone` / `pull` / `push` require an `https://` URL; plain `http://` is accepted only for a loopback host (local development). Other schemes are rejected.

21. Remotes: Push and Pull over HTTPS

A remote is an HTTPS endpoint that stores and serves a store's module and accepts authorized updates. A remote is also a serving provider and inherits provider blindness (§15).

21.1 Overview

A client configures a remote with `digstore remote add <name> <url>` and thereafter clones, pulls, and pushes against it.

21.2 REST Surface

Method and path	Purpose
<code>GET /stores/{storeID}</code>	Store descriptor: current root, size, public key, served-root signature
<code>GET /stores/{storeID}/roots</code>	Root history
<code>HEAD /stores/{storeID}/module</code>	Existence, size, and ETag (the current root)
<code>GET /stores/{storeID}/module</code>	Download the module (<code>application/wasm</code>)
<code>PUT /stores/{storeID}/module</code>	Push a new module or delta (authorized)
<code>POST /stores/{storeID}/content</code>	Retrieve content by retrieval key, root, and range
<code>POST /stores/{storeID}/proof</code>	Retrieve a proof for a retrieval key and root
<code>GET /stores/{storeID}/delta?from=&to=</code>	Chunk delta between two generations
<code>POST /stores/{storeID}/delta</code>	Negotiated delta from a client have-summary

21.3 Clone and Fetch — Module Verification

`clone` downloads the module via `GET /module` and **verifies it locally before installing or executing it**. `fetch` retrieves the descriptor and root history to learn whether the remote holds a newer generation, without downloading content.

A downloaded module is verified, purely from its own bytes and the descriptor, before it is trusted:

1. **Self-certifying identity.** The module's embedded `StoreId` must equal the requested store id, and `SHA-256(embedded PublicKey) == StoreId` (§4.2). A module for a different store, or one whose

embedded key does not hash to the id, is rejected.

2. **Content-root consistency.** The merkle root recomputed from the module's own embedded leaves must equal both the embedded `CurrentRoot` and the served root advertised by the descriptor/ETag.
3. **Authenticated head (§21.6).** The served root must carry a valid publisher BLS signature, verified against the embedded (now-trusted) store key.

Steps 1–2 prove the module is a self-consistent build for the requested identity; step 3 proves the served root was *authorized by the publisher's private key*, so a malicious origin that holds only the public store key cannot serve fabricated content. A clone fails closed if any check fails — including an absent signature, so a server cannot strip the signature to downgrade the check.

21.4 Pull and Head Advancement

`pull` brings the local store up to the remote's current head, downloading the module (or a delta, §21.5) for the newer generation and verifying it with the same checks as clone (including the authenticated-head signature). The remote's head is the generation it currently serves.

A remote may decouple acceptance of a push from advancement of the served head: it MAY accept a push into a pending/staged state and defer advancing the head until an external authorization completes. A pending generation is not served; until the head advances, the remote continues to serve its last confirmed generation, and the confirmed head's signature is the authoritative one a client verifies.

21.5 Delta Sync

Rather than transfer a whole module, a client and remote can exchange only the chunks that differ. `GET /delta?from=&to=` returns the chunks present in the target generation and absent from the source along a linear ancestry; `POST /delta` negotiates a delta from a client-supplied have-summary. Because chunks are content-addressed and deduplicated (§8), the delta is exactly the new chunk set plus the key-table changes. **The client verifies every delta chunk against its advertised content address (`SHA-256(chunk) == hash`)** before accepting it, so a server cannot substitute chunk bytes.

21.6 Push and the Authenticated Head

A push uploads a new module (or delta) through `PUT /module`. The push is authorized by a signature: the publisher signs the canonical message `SHA-256(root || store_id)` with the store's BLS key, and the remote verifies that signature against the store's public key. A remote MAY additionally require a bearer token alongside the BLS signature. A push must be a fast-forward of the remote's current head; a non-fast-forward push is rejected (§21.8).

Authenticated head. The remote *persistent* the verified push signature for each root and returns the signature for the served head in the store descriptor (`GET /stores/{id}`). On `clone / pull`, the client re-verifies that signature against the store-id-bound public key embedded in the module (§21.3). This closes the gap between "self-consistent module" and "publisher-authorized content": the served root is provably one the publisher signed, not merely one a server fabricated consistently. (A genesis root that a remote was

seeded with out of band and never received via a signed push has no signature; such a head fails the clone check by design — a real publisher reaches a remote by pushing, which is signed.)

21.7 ETags and Caching

The module's ETag is its root. A client holding a generation issues a conditional request with `If-None-Match` and receives `304 Not Modified` when its root matches the remote's head. Because content at a root is immutable, cached module and content responses remain valid until the root changes; a new generation is a new root and a new cache identity.

21.8 Status Codes

Code	Meaning
200	Content (real or decoy), descriptor, or module bytes
201	Push accepted; a new generation is now the head
202	Push accepted into a pending state; head not yet advanced (§21.4)
304	<code>If-None-Match</code> matched the current root
401 / 403	Push not authorized: missing bearer token, or bad/invalid BLS signature
404	Unknown store, or unknown root for a module or descriptor. Never for content.
409	Non-fast-forward push: the pushed parent is not the current head
413 / 422	Module exceeds the size limit, or failed validation / malformed headers
429	Rate limited

Rate limiting. The remote applies a per-store token-bucket rate limiter that refills over time, so a one-time request burst cannot permanently lock out a store, and the bucket map is bounded so attacker-chosen store ids cannot exhaust memory. Internal (5xx) error bodies do not leak server-internal detail such as filesystem paths.

Part 5: Properties and References

22. Security Considerations and Residual Risks

A summary of what the format guarantees, what it depends on, and what remains open.

Property	Rests on
Confidentiality of content	URN secrecy; HKDF-SHA256 key derivation; AES-256-GCM-SIV (§11)
Integrity of responses	SHA-256 merkle proofs bound to a trusted root (§9)
Authenticated encryption	AES-256-GCM-SIV tags; nonce-misuse resistance (§11.2)
Authenticated head	Publisher BLS signature over the served root, verified on clone/pull (§21.6)
Provider blindness	Retrieval and decryption keys are functions of a URN the provider never holds (§15)
Indistinguishability	Deterministic decoys and oblivious access (§14)
Host trust	BLS attestation against compile-time trusted keys (§12)
Secretlessness	No decryption key, signing key, or proving secret in the module (§17)

These properties hold as long as the standard primitives remain secure: SHA-256, AES-256-GCM-SIV, HKDF-SHA256, BLS signatures, and (when enabled) the zero-knowledge proof system. The URN is a capability, so confidentiality is contingent on URNs being kept secret by those they are shared with.

Residual risks (open hardening items). This edition is explicit about what is not yet closed:

- **R1 — Pending-head and revocation semantics.** Authenticated-head verification (§21.6) binds the *served* root to a publisher signature, but the format does not yet define key rotation or root revocation: a leaked store key cannot be rotated without a new store id, and there is no signed "tombstone" to retract a previously published root. Treat the store key as long-lived.
- **R2 — Domain separation (merkle and BLS).** Neither the merkle tree nor the BLS signing messages yet carry explicit domain-separation tags (§9.1, §12.3). In practice the relevant messages are already structurally distinguished — merkle leaves hash a variable-length ciphertext blob while interior nodes hash exactly 64 bytes; the three BLS roles bind different fields and lengths and the trusted root is verifier-supplied — so the practical exposure is low. Adding RFC-6962-style leaf/node tags and per-role BLS prefixes is recommended defense-in-depth; both are breaking changes (every root / every signature) and would be made together in one format-version bump.
- **R3 — Execution-proof backend.** The default proof backend is a forgeable mock with a fixed clock and mock chain source (§13.1). Execution proofs carry no cryptographic weight until the RISCO backend, a real chain source, and a real clock are enabled. Content integrity does not depend on proofs (it rests on §9 and §21.6).
- **R4 — JWT signature verification.** The optional JWT gate checks claims but does not yet verify token signatures, and is disabled by default (§6.3). JWT authorization must not be relied upon until signature verification is enforced on the content path under a deterministic, proof-compatible design.
- **R5 — Oblivious-access strength.** The in-module oblivious access pattern (§14.3) decorrelates pool access from the request, but its resistance to a sophisticated statistical/timing adversary depends on the padding and reordering policy and is an area of ongoing hardening. Decoys and provider blindness do not depend on it.

23. Acknowledgements and References

Digstore stands on established work. Content-defined chunking follows the rsync and FastCDC line with a gear-based rolling hash. Merkle commitments follow Git and Ethereum. Authenticated encryption uses AES-256-GCM-SIV (RFC 8452); key derivation uses HKDF-SHA256. Execution and sandboxing build on WebAssembly and wasmtime-class runtimes. Signatures and host attestation use BLS over BLS12-381, as in the Chia ecosystem. Execution proofs draw on the zkVM and proof-carrying-code literature.

Concern	Crate
WASM execution	<code>wasmtime</code>
WASM emission and parsing	<code>wasm-encoder</code> , <code>wasmparser</code>
Hashing	<code>sha2</code>
Authenticated encryption	<code>aes-gcm-siv</code>
Key derivation	<code>hkdf</code>
Signatures (BLS)	<code>chia-bls</code> (blst)
Content-defined chunking	gear-based CDC
Zero-knowledge proofs	RISCo zkVM (<code>risc0</code>)

This document specifies the local store format and its HTTPS remote protocol. Network distribution across many providers, external identity anchoring of the store ID, and payment settlement are out of scope and are addressed by separate DIG Network specifications.

24. Changes from v1.0

This edition reconciles the specification with the audited, hardened implementation. Substantive changes:

- **v2.1 — multi-store workspaces and limits (§4.4, §5.1, §8.3, §18.2).** `.dig` is now a workspace of named stores (`stores/<name>/` , `workspace.toml` registry + active selection + per-store content root); each store has a per-store **128 MB content cap**; size-obfuscation filler is now **uniform** so every module is one fixed size (the module size reveals nothing about content size, replacing the earlier coarse-bucket framing); and the module memory ceiling is raised to a configurable **384 MiB** with the guest heap placed dynamically above the data section so contract D2 holds for any blob size.
- **Chunk cipher (§11.2).** v1.0 specified AES-256-GCM with a fixed nonce and argued it was safe "because the key is unique per URN." That rationale is unsound — a rootless URN reuses its key across generations, recreating the catastrophic key+nonce reuse that GCM forbids. v2.0 specifies **AES-256-GCM-SIV** (nonce-misuse resistant) and gives the correct rationale.
- **Merkle leaves (§9.1).** v1.0 said "a leaf is `SHA-256(chunk)` ." The implementation commits **per-resource ciphertext digests** (`SHA-256` of a resource's concatenated ciphertext chunks); v2.0

documents this, and proof size is logarithmic in the number of resources.

- **Authenticated head (§21.3, §21.6).** New. Clone/pull now verify a publisher BLS signature over the served root, persisted by the remote and returned in the descriptor. This upgrades the integrity claim of §1 from "self-consistent module" to "publisher-authorized content."
- **Data-section endianness (§5.2).** Corrected from "little-endian" to the implementation's **big-endian** (Chia streamable) framing; the canonical encoding is now documented.
- **Execution proofs (§13.1).** Restated to describe what the circuit actually proves (re-execution of the deterministic serving computation, with `program_hash` as an identifier) and to note that the default backend is a mock.
- **JWT (§6.3).** Restated as a claims-only gate that does not yet verify signatures and is off by default — not the signature-verifying gate v1.0 implied.
- **BLS signing roles (§12.3).** Documented the three signing roles and their exact messages, and flagged per-role domain-separation tags as a recommended hardening (§22 R2) rather than implying they are already applied.
- **Hardening surfaced in the spec.** OS-CSPRNG key generation (§20.1), owner-only key files (§4.1), host memory/table/fuel/timeout bounds and disabled threads (§18.2), `jwt_fetch` SSRF guard (§6.3), `checkout` path-traversal guard (§20.2), HTTPS-only remotes (§20.2), delta-chunk integrity (§21.5), time-based rate limiting and non-leaking errors (§21.8).
- **Residual risks (§22).** New section, mirroring the project's `SECURITY.md`.
- **Formatting.** Tables that were garbled in extraction (positioning, heritage, URN components, proof structure, REST surface, status codes) are corrected and aligned.

25. Glossary

- **Store ID** — 32-byte `SHA-256` of the store's BLS public key; the self-certifying store identifier.
- **Root hash / root** — merkle root over a generation's per-resource leaves; the commit identifier.
- **Program hash** — `SHA-256` of the compiled module bytes; identifies a build. Advances on any byte change (including a metadata edit), independent of the content root.
- **Generation** — one committed store state: a chunk set, a merkle tree, a root.
- **Retrieval key** — `SHA-256(canonical_urn)` ; the address a resource is located under.
- **Decryption key** — `HKDF-SHA256(canonical_urn[, salt])` ; the AES-256-GCM-SIV key for a resource's chunks.
- **Authenticated head** — the property that a remote's served root carries a publisher BLS signature a client verifies on clone/pull (§21.6).
- **Decoy** — deterministic bytes returned for a retrieval miss, indistinguishable on the wire from a real response (§14.2).
- **Neutral pipe / provider blindness** — a serving node holds only ciphertext keyed by hashes and cannot read what it serves (§15).